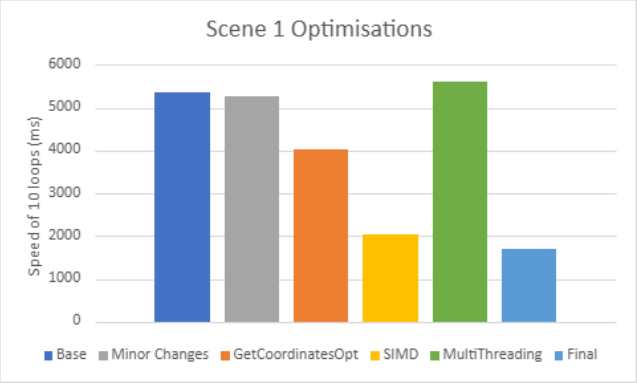
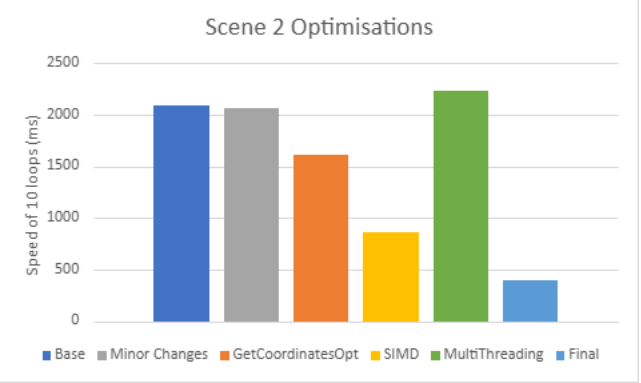
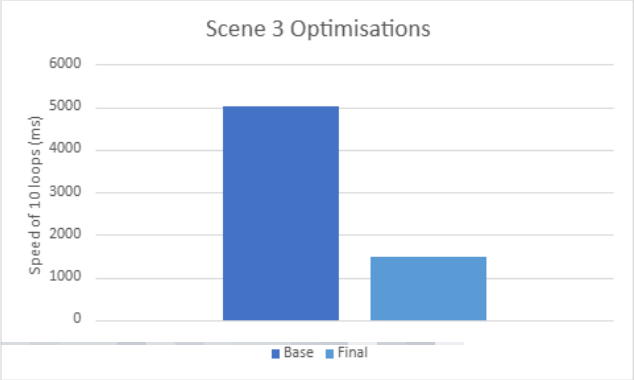
Optimised CPU Rasterizer

Section 1:

Through this part of the assignment, I worked to implement technologies and concepts to optimise a given rasteriser to produce improved performance for 3 scenes. On initial testing of the base rasterizer running each scene for 10 loops, scene 1 got an average of 5407.17ms per loop with a max variation of 4.91% over 50 total loops. Scene 2 got 2125.08ms with a max variation of 6.20%. My final optimisations were able to achieve a global improvement of: Scene 1 – 3.14x speed, Scene 2 – 5.24x speed, Scene 3 – 3.41x speed.







The Optimisations I added were minor algorithmic and structural changes, Cache organisation and early exiting when getting each pixel coordinate, using single instruction multiple data (SIMD) in the draw, get coordinates, and get C, and finally adding multi-threading to the render function.

The settings I used throughout my project are shown in figure 4 and 5 and show that I used C++ 20 standard and AVX2. These settings allowed me to include jthreads which made the joining of threads much easier and AVX2 which allowed me to implement SIMD correctly.

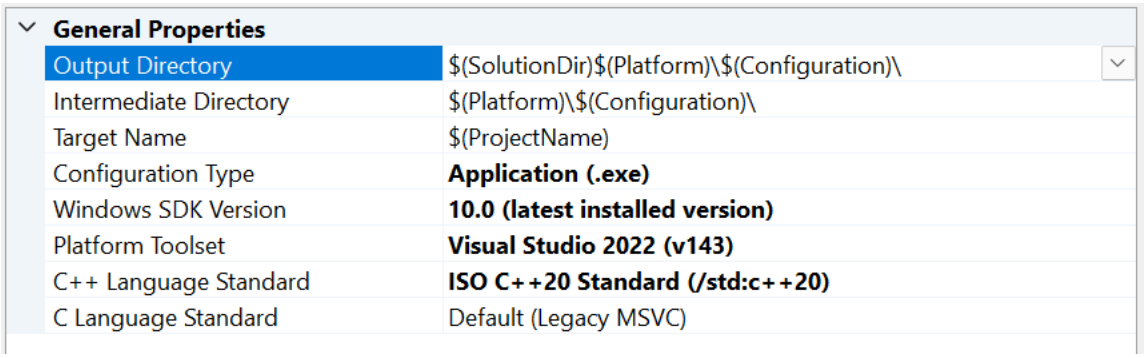


Figure 4: General settings showing C++ 20 use

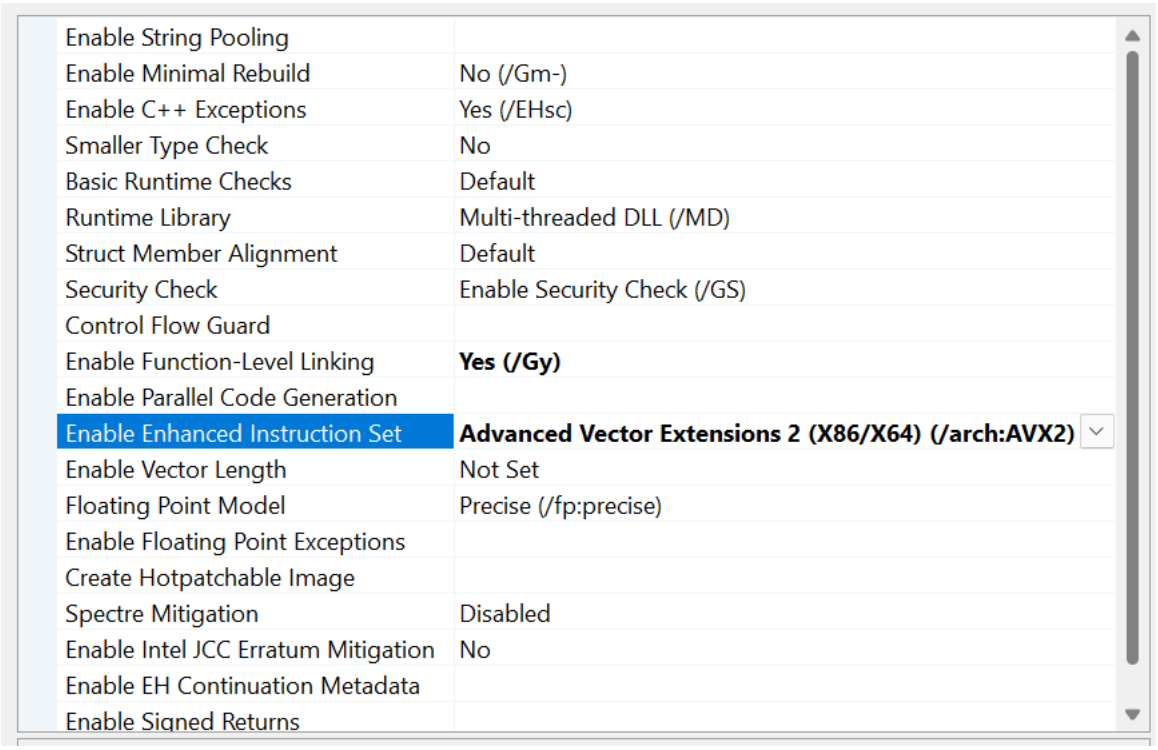


Figure 5: Code generation settings show AVX2 use

Section 2:

After reading through and understanding the code as best as possible, I went through it to try and change parts that could make some impact when the changes were added together. These included unrolling the matrix multiplication, reducing the number of variable creations where possible, and reducing the number of divisions on busy calls, such as in getCoordinates.

```
alpha = getC(vec2D(v[0].p), vec2D(v[1].p), p) / area;
beta = getC(vec2D(v[1].p), vec2D(v[2].p), p) / area;
gamma = getC(vec2D(v[2].p), vec2D(v[0].p), p) / area;

const float invArea = 1.0f / area;
```

```
alpha = getC(vec2D(v[0].p), vec2D(v[1].p), p) * area;
beta = getC(vec2D(v[1].p), vec2D(v[2].p), p) * area;
gamma = getC(vec2D(v[2].p), vec2D(v[0].p), p) * area;

//unrolled Matrix matrix multiplication
 matrix operator * (const matrix& mx) const {
     matrix ret;
     ret.a[0 * 4 + 0] =
         a[0 * 4 + 0] * mx.a[0 * 4 + 0] +
         a[0 * 4 + 1] * mx.a[1 * 4 + 0] +
         a[0 * 4 + 2] * mx.a[2 * 4 + 0] +
         a[0 * 4 + 3] * mx.a[3 * 4 + 0];
     ret.a[1 * 4 + 0] =
         a[1 * 4 + 0] * mx.a[0 * 4 + 0] +
         a[1 * 4 + 1] * mx.a[1 * 4 + 0] +
         a[1 * 4 + 2] * mx.a[2 * 4 + 0] +
         a[1 * 4 + 3] * mx.a[3 * 4 + 0];
     ret.a[2 * 4 + 0] =
         a[2 * 4 + 0] * mx.a[0 * 4 + 0] +
         a[2 * 4 + 1] * mx.a[1 * 4 + 0] +
         a[2 * 4 + 2] * mx.a[2 * 4 + 0] +
         a[2 * 4 + 3] * mx.a[3 * 4 + 0];
     ret.a[3 * 4 + 0] =
         a[3 * 4 + 0] * mx.a[0 * 4 + 0] +
         a[3 * 4 + 1] * mx.a[1 * 4 + 0] +
         a[3 * 4 + 2] * mx.a[2 * 4 + 0] +
         a[3 * 4 + 3] * mx.a[3 * 4 + 0];
     ret.a[0 * 4 + 1] =
         a[0 * 4 + 0] * mx.a[0 * 4 + 1] +
```

```
void render(Renderer& renderer, Mesh* mesh, matrix& camera, Light& L) {
    // Combine perspective, camera, and world transformations for the mesh
    matrix p = renderer.perspective * camera * mesh->world;
    float width = static_cast<float>(renderer.canvas.getWidth());
    float height = static_cast<float>(renderer.canvas.getHeight());

    // Iterate through all triangles in the mesh
    for (triIndices& ind : mesh->triangles) {
        Vertex t[3]; // Temporary array to store transformed triangle vertices

        // Transform each vertex of the triangle
        for (unsigned int i = 0; i < 3; i++) {
            t[i].p = p * mesh->vertices[ind.v[i]].p; // Apply transformations
            t[i].p.divideW(); // Perspective division to normalize coordinates

            // Transform normals into world space for accurate lighting
            // no need for perspective correction as no shearing or non-uniform
            t[i].normal = mesh->world * mesh->vertices[ind.v[i]].normal;
            t[i].normal.normalise();

            // Map normalized device coordinates to screen space
            t[i].p[0] = (t[i].p[0] + 1.f) * 0.5f * width;
            t[i].p[1] = (t[i].p[1] + 1.f) * 0.5f * height;
            t[i].p[1] = height - t[i].p[1]; // Invert y-axis

            // Copy vertex colours
            t[i].rgb = mesh->vertices[ind.v[i]].rgb;
        }
```

```
static matrix makeRotateXYZ(float x, float y, float z) {
    float sinx = std::sin(x);
    float cosx = std::cos(x);
    float siny = std::sin(y);
    float cosy = std::cos(y);
    float sinz = std::sin(z);
    float cosz = std::cos(z);
    return matrix::makeRotateX(sinx, cosx) *
        matrix::makeRotateY(siny, cosy) *
        matrix::makeRotateZ(sinz, cosz);
}
```

Next, I used the performance profiler to find where the majority of the cpu cost was. I found that the getCoordinates function inside the draw call was the highest usage. Scene 1 had 24,758 (41.36%) calls, and scene 2 had 10,228 (45.32%) calls.

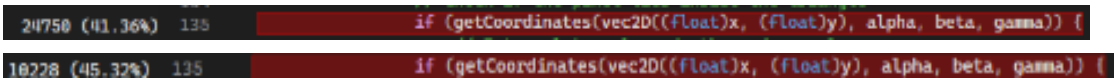


Figure 7: getCoordinates cpu usage on 10 loops for scene 1 and 2

I recognised that there were multiple creations of the same vectors, and that I could split up the detection of negative values to create an early exit.

```
bool getCoordinatesOptimised(vec2D& p, float& alpha, float& beta, float& gamma) {
    //cache each point once
    const vec2D v0p = (vec2D(v[0].p));
    const vec2D v1p = (vec2D(v[1].p));
    const vec2D v2p = (vec2D(v[2].p));
```

```
    //multiply rather than divide
    const float invArea = 1.0f / area;

    //early exit if possible
    alpha = getC(v0p, v1p, p) * invArea;
    if (alpha < 0.f) return false;
    beta = getC(v1p, v2p, p) * invArea;
    if (beta < 0.f) return false;
    gamma = getC(v2p, v0p, p) * invArea;
    if (gamma < 0.f) return false;

    return true;
}
```

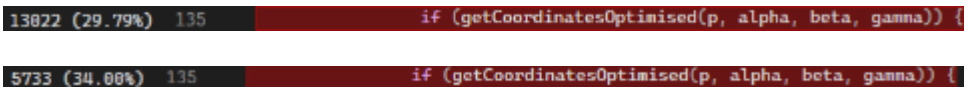


Figure 8: getCoordinatesOptimised cpu usage on 10 loops for scene 1 and 2

This change immediately improved performance by 25% reducing the number of calls to almost 50%.

Section 3:

I then implemented SIMD in the rendering draw call which made a substantial difference. While writing this code I decided to include the minor changes and getCoordinates optimisations as this reduced the amount of comparisons I would need to make within my code. The introduction of SIMD into the draw function provided a boost of 60% speed from the base rasterizer.

```
__m256 getCoordinatesSIMD(__m256& vx, __m256& vy, __m256& alpha, __m256& beta,
__m256& gamma) {

    alpha = getCSIMD(vec2D(v[0].p), vec2D(v[1].p), vx, vy);
    beta = getCSIMD(vec2D(v[1].p), vec2D(v[2].p), vx, vy);
    gamma = getCSIMD(vec2D(v[2].p), vec2D(v[0].p), vx, vy);
    __m256 va = _mm256_set1_ps(area);

    alpha = _mm256_div_ps(alpha, va);
    beta = _mm256_div_ps(beta, va);
    gamma = _mm256_div_ps(gamma, va);
    __m256 zero = _mm256_setzero_ps();

    __m256 test_alpha = _mm256_cmp_ps(alpha, zero, _CMP_GE_OQ);
    __m256 test_beta = _mm256_cmp_ps(beta, zero, _CMP_GE_OQ);
    __m256 test_gamma = _mm256_cmp_ps(gamma, zero, _CMP_GE_OQ);

    __m256 inside_mask = _mm256_and_ps(_mm256_and_ps(test_alpha, test_beta),
test_gamma);

    return inside_mask;
}
```

I also needed to add a helper function to interpolate using the SIMD syntaxing. Due to a quirk in the code, the interpolating calculation needed to be beta, gamma, alpha to make sure the colour and lighting were calculated correctly.

```
__m256 interpolateSIMD(__m256 alpha, __m256 beta, __m256 gamma, __m256 a1, __m256
a2, __m256 a3) {
    return _mm256_add_ps(_mm256_add_ps(_mm256_mul_ps(a1, beta), _mm256_mul_ps(a2,
gamma)), _mm256_mul_ps(a3, alpha)
    );
}
```

I decided to add multi-threading to the draw call inside the render function as I thought this was the most sensible place to parallelise the rasterizer. I decided to start by testing the speeds of each by setting each thread manually. I found that 1 thread per 150 triangles seemed to provide the best results, in addition, I implemented a check to use a non-multi-thread version to draw calls that were less than 200 triangles. After finding the possible ideal triangles per thread, I decided to make a simple scene which contained enough triangles to fill each thread fully. The scene I decided on was a grid of 15 by 18 cubes which contained 3240 triangles. Each cube spun randomly and the camera moved to each side with some of the cubes going off screen. With a thread count of 22 this should allow each thread to manage 147 triangles.

```
//get number of triangles to check if MT needed
size_t triangleCount = jobs.size();

if (triangleCount < 200) {
    for (auto& job : jobs) {
        job.tri.drawSIMDOptimised(renderer, L, job.ka, job.kd);
    }
    return;
}

//after testing - ideal threads for this code is around 150 triangles per 1
thread
unsigned int numThreads = std::min((unsigned
int)std::jthread::hardware_concurrency(), (unsigned int) triangleCount/150);
std::vector<std::jthread> threads(numThreads);

for (unsigned int i = 0; i < numThreads; i++) {
    threads[i] = std::jthread([i, numThreads, &jobs, &renderer, &L]() {
        // Each thread processes every Nth triangle
        for (size_t j = i; j < jobs.size(); j += numThreads) {
            jobs[j].tri.drawSIMDOptimised(renderer, L, jobs[j].ka, jobs[j].kd);
        }
    });
}
```

I recognised that with my initial method of multi-threading I was creating and destroying the threads every frame which added an expense cost to the speed of the scenes. I added my next optimisations of pooling and stripping in steps, recording how each number of threads performed. I was expecting to see a general increase in speed over the optimisations with hopefully a flatter curve when increasing thread count. This didn't show as expected and I found the results to be unpredictable. The particular interest was when adding multi-thread pooling, I got a large increase in speed at 8 threads for scene 2, and scene 3 became globally much worse. The other

outliers were 3 threads when multi-threading with a strip allocation and 16 threads for scene 2 on multi-thread polling and stripping being surprisingly fast.

```
for (unsigned int i = 0; i < numThreads; i++) {
    threads[i] = std::jthread([i, numThreads, &jobs, &renderer, &L]() {
        for (size_t j = i; j < jobs.size(); j += numThreads) {
            jobs[j].tri.drawSIMDOptimised(renderer, L, jobs[j].ka, jobs[j].kd);
        }
        });
}
```

*Multi-threading

```
for (unsigned int i = 0; i < numThreads; i++) {
    threads[i] = std::jthread([i, stripHeight, numThreads, canvasHeight, &jobs,
&renderer, &L]() {
        int yMin = i * stripHeight;
        int yMax = (i == numThreads - 1) ? canvasHeight : (i + 1) * stripHeight;

        for (auto& job : jobs)
            job.tri.drawSIMDOptimisedStrip(renderer, L, job.ka, job.kd, yMin,
yMax);
    });
}
```

*Multi-threading with strip allocation

```
for (int i = 0; i < numThreads; i++) {
    pool.enqueue([i, &jobs, &renderer, &L]() mutable {
        for(auto& job : jobs)
            job.tri.drawSIMDOptimised(renderer, L, job.ka, job.kd) ;
        });
}
pool.waitForCompletion();
```

*Multi-threading with thread pooling

```
for (int i = 0; i < numThreads; i++) {
     pool.enqueue([i, stripHeight, numThreads, canvasHeight, &jobs, &renderer,
&L]() mutable {
         int yMin = i * stripHeight;
         int yMax = (i == numThreads - 1) ? canvasHeight : (i + 1) * stripHeight;
         for(auto& job : jobs)
             job.tri.drawSIMDOptimisedStrip(renderer, L, job.ka, job.kd, yMin,
yMax) ;
         });
 }
 pool.waitForCompletion();
```

*Multi-threading with thread pooling and strip allocation

```
int yMin = i * stripHeight;
int yMax = (i == numThreads - 1) ? canvasHeight : (i + 1) * stripHeight;
for (auto& job : jobs)     job.tri.drawSIMDOptimisedStrip(renderer, L, job.ka,
job.kd, yMin, yMax);

//inside draw strip function
int startY = std::max((int)(minV.y), yMin);
int endY = std::min((int)ceil(maxV.y), yMax);
for (int y = startY; y < endY; y++) {
     int x = (int)(minV.x);
     int maxX = (int)ceil(maxV.x);
```

```
for (; x < maxX; x += 8) {
```

*Strip allocation code

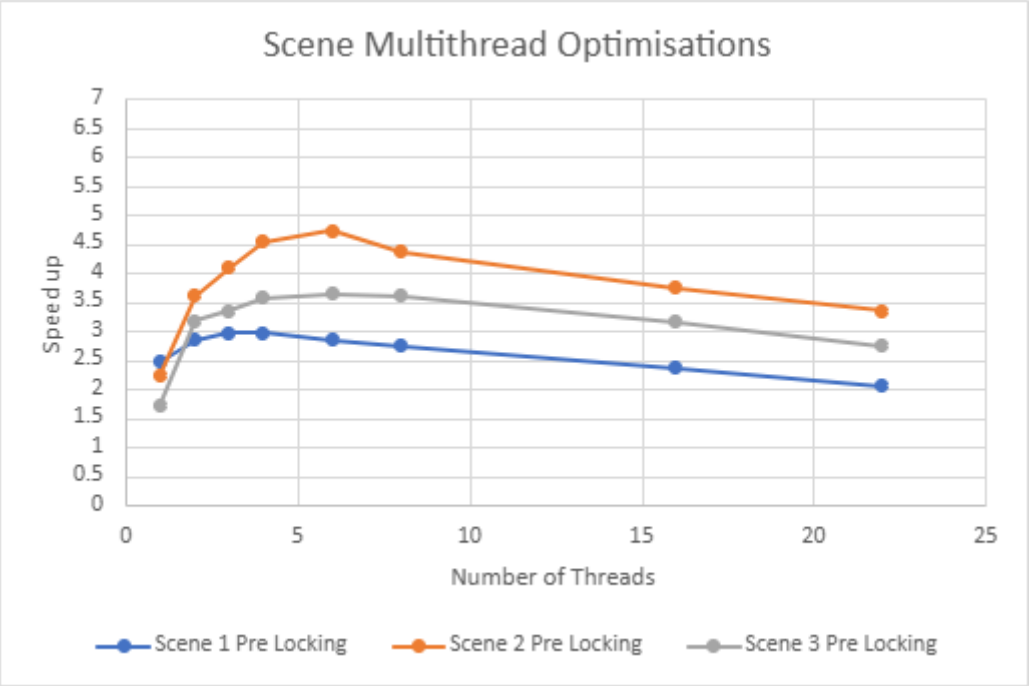


Figure 9:Multi-thread speed up graph

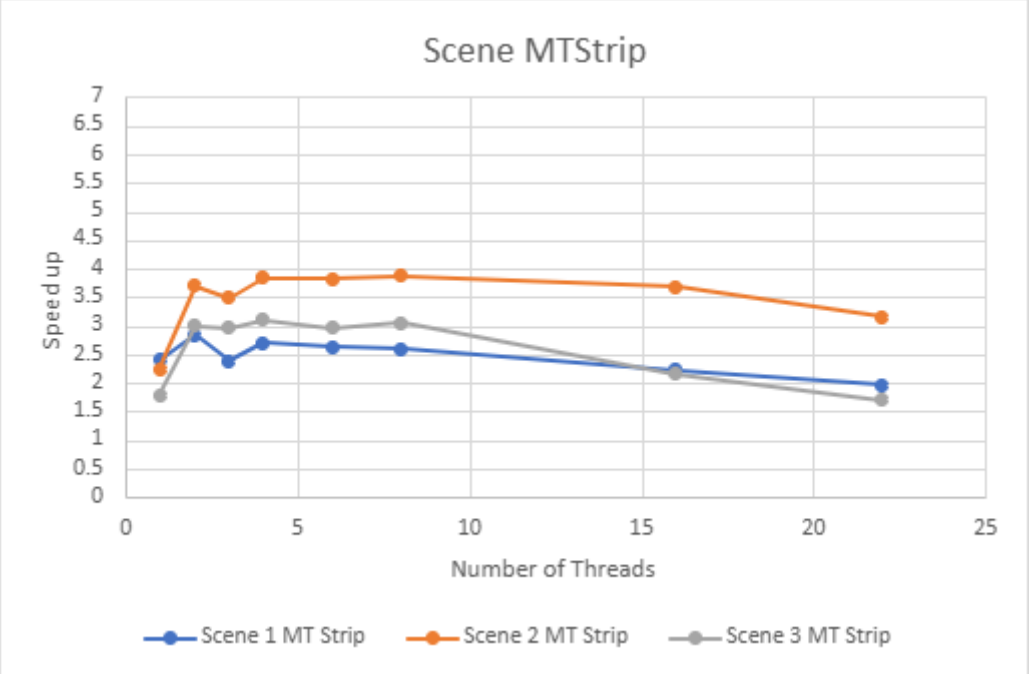


Figure 10:Multi-thread and strip allocation speed up graph

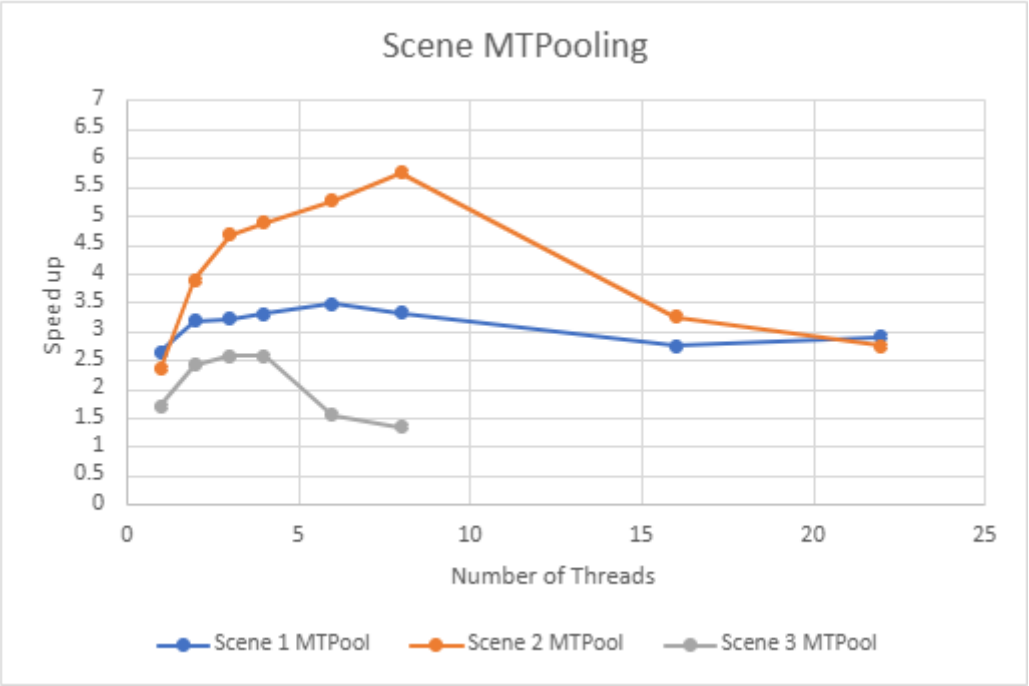


Figure 11: Multi-thread with thread pooling speed up graph

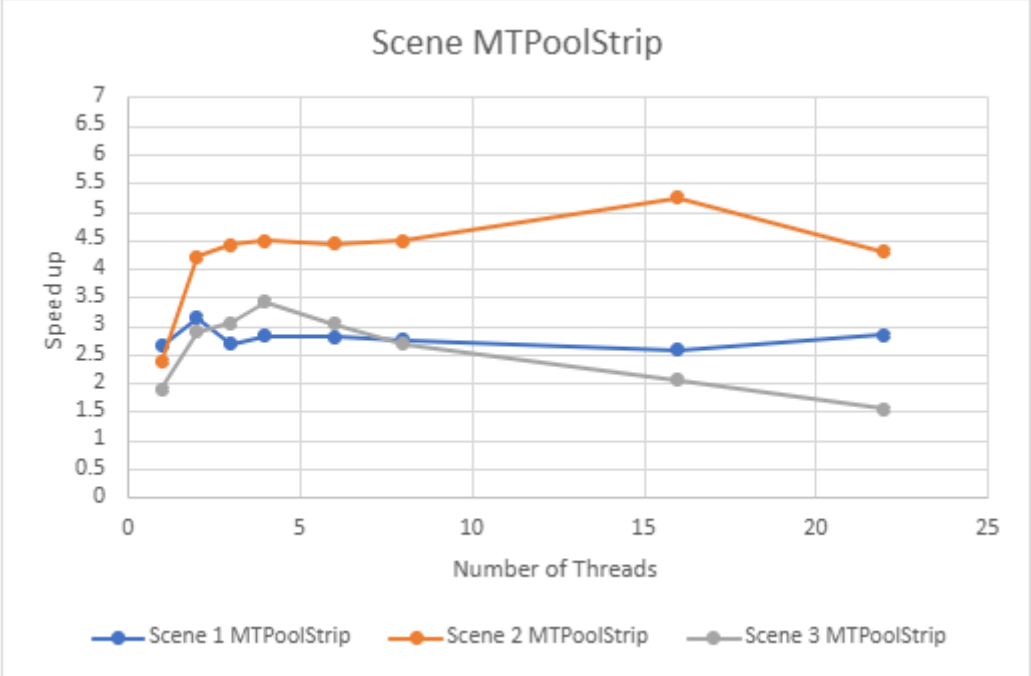


Figure 12:Multi-thread with thread pooling and strip allocation speed up graph

Section 4:

Given the time frame I am pleased with the optimisations I implemented and how they affected the scene, achieving 3.14x speed for scene 1, 5.24x speed for scene 2, and 3.41x speed for scene 3. Adding SIMD alone was the largest improvement I achieved in one go over all scenes. This will be a key optimisation I include in future code. While adding multi-threading to scene 3 gave substantial speed ups, more than in scene 1, I didn't achieve the bonuses I was expecting to. I believe this is due to the wait for completion function that I implemented which waits for all threads to line up appropriately. While adding the strip allocation to my multi-thread implementation created a more level speed up graph, it only improved the higher thread counts up to a similar level rather than levelling out and boosting speeds. Figure 9 versus figure 10. I need to do more reading on multi-threading to truly understand where the limitations hide and what I can do to manage the bottle necks where they come up. Part way through this project I realised a more concrete approach to testing and implementing optimisation tests is to run the program on a fresh restart with only the necessary application open. In this case that would be the IDE visual studio. This way you can limit the other variables on the machine that may impact performance.